



Facultad de
Ciencias

UNAM

Proyecto 1

Reconocimiento de Figuras

Profesor: José de Jesús Galaviz Casas
Ayudante: Brenda Ayala Flores
Ayudante Lab: José Eduardo Cruz Campos

Modelado y Programación
Grupo 7078, Semestre 2027-1
13 de septiembre de 2026



Índice

Proyecto 1	1
1.1. Consideraciones generales	1
1.2. Objetivo	1
1.3. Supuestos sobre las imágenes de prueba	1
1.4. Qué debe hacer el programa	2
1.5. Reporte técnico	2
1.6. Reporte de uso de algún modelo de lenguaje	2
1.7. Fecha de Entrega	3
1.8. Entregables	3
1.9. Rúbrica	4

Proyecto 1

Planteamiento del Problema

Dada una imagen .bmp que contiene una o más figuras geométricas de colores sólidos (distintos entre sí y del fondo, sin degradados ni suavizado en los bordes), el programa debe identificar cada figura y reportar su color en hexadecimal y su categoría:

- **C** — Cuadriláteros (cuadrados, rectángulos, rombos, trapezoides)
- **T** — Triángulos (equiláteros, isósceles, escalenos, rectángulos)
- **O** — Círculos
- **X** — Otros (cualquier figura que no caiga en las anteriores)

1.1. Consideraciones generales

- **Equipos:** el proyecto se realiza en equipos de 3 a 4 personas. No se aceptarán entregas individuales.
- **Lenguaje de programación:** es libre, pueden usar el lenguaje de su preferencia, *siempre y cuando justifiquen su elección* en la sección “Arsenal” del reporte técnico (no basta con decir cuál usaron; deben explicar por qué).
- **Funcionamiento mínimo:** el programa debe funcionar correctamente ejecutándose desde terminal (línea de comandos). No se requiere interfaz gráfica para la funcionalidad mínima.
- Sube a la asignación de Classroom un README.md inicial con el nombre del equipo y sus integrantes (Solo un integrante del equipo debe subir este readme).

1.2. Objetivo

Construir una versión funcional mínima capaz de leer una imagen .bmp, encontrar las figuras que contiene y clasificarlas correctamente en C/T/O/X, reportando el resultado al usuario.

1.3. Supuestos sobre las imágenes de prueba

- El fondo es de un solo color, uniforme.
- Cada figura tiene un color sólido, distinto entre figuras y distinto del fondo.
- No hay anti-aliasing ni colores intermedios en los bordes.
- Las figuras no se traslapan entre sí.
- Tamaño, rotación y posición de las figuras son arbitrarios.

El equipo deberá generar o conseguir su propio banco de imágenes de prueba — no se entregará uno; esto es intencional: forma parte de validar su propia solución. Su banco de imágenes debe contener **mínimo 10 imágenes**.

1.4. Qué debe hacer el programa

Tu equipo debe diseñar un programa que, a partir de la ruta de una imagen, sea capaz de identificar cada figura presente, determinar su color y clasificarla correctamente en una de las cuatro categorías definidas, reportando el resultado de forma clara para quien lo use. Más allá de que funcione, considera qué características hacen que tu solución sea confiable y utilizable: cómo se comporta ante una entrada inválida o mal escrita (¿el programa avisa con claridad, o truena con un error críptico?), qué tan rápido procesa una imagen de tamaño razonable, y qué tan clara y mantenible es la implementación.

Como parte de su reporte técnico (sección de Análisis del problema, más abajo), el equipo debe **identificar y enumerar explícitamente** cuáles son los requisitos funcionales y no funcionales de su propia solución. Esa identificación es en sí misma parte de lo que se evalúa.

1.5. Reporte técnico

Un documento breve (no es necesario que sea extenso) con:

1. **Definición del problema** — qué entrada reciben, qué deben producir.
2. **Arsenal** — lenguaje y bibliotecas elegidas, y *por qué* (no solo cuáles).
3. **Análisis del problema** — supuestos, casos límite que identificaron, y el desglose de requisitos funcionales y no funcionales mencionado arriba.
4. **Selección de la mejor alternativa** — el algoritmo que decidieron usar para detectar y clasificar figuras, y por qué lo prefirieron sobre otras opciones que consideraron.
5. **Diagrama de flujo o pseudocódigo** de la solución.

1.6. Reporte de uso de algún modelo de lenguaje

Nota: pedir que documenten el uso de algún modelo de lenguaje no es una invitación a delegar el desarrollo del proyecto en estas herramientas, ni significa que el desarrollo del programa deba hacerse exclusivamente a través de ellas. Estas herramientas son un apoyo: úsenlas con criterio y asegúrense de comprender y poder justificar aquello que entregan. Si se detecta que el equipo no comprende su propio proyecto, por ejemplo, si no puede explicar el algoritmo que implementó o el código entregado, el proyecto quedará **ANULADO**. :(

Documento aparte, **obligatorio**, donde documentan su uso de herramientas como algún modelo de lenguaje durante el desarrollo. Para que no se vuelva un trámite vacío, debe incluir, por cada prompt relevante:

- El **prompt** que usaron (tal cual, o resumido si fue muy largo).
- ¿Por qué decidieron usar este tipo de herramienta?
- Qué parte de la respuesta **aceptaron tal cual**.
- Qué parte **corrigieron o descartaron**, y por qué.
- **Al menos un caso** en que el modelo de lenguaje se equivocó o dio una solución que no funcionaba, y cómo lo detectaron.

Adicionalmente, como control de que el equipo entiende lo que entrega: deben poder explicar, **sin apoyo del modelo de lenguaje**, el algoritmo que implementaron para clasificar las figuras.

1.7. Fecha de Entrega

24 Septiembre del año en curso.

1.8. Entregables

El equipo debe trabajar sobre un único repositorio (GitHub) para todo el proyecto. No se les da una estructura de carpetas predefinida: parte de lo que se evalúa es que sepan organizar su código, pruebas y documentación de forma clara y ordenada. Todos los integrantes deben participar activamente en el repositorio (se espera ver aportaciones de cada uno). Además, deberán agregarme como colaboradora del repositorio. La entrega en Classroom consiste únicamente en la liga (URL) al repositorio, y **debe subirla la misma persona que registró al equipo**.

1. Código fuente, en el lenguaje que elijan
2. Imágenes de prueba usadas + resultados esperados
3. README
4. Reporte Técnico
5. Reporte Modelo de Lenguaje (ver sección 5)

1.9. Rúbrica

Rubro	Peso
Funcionalidad mínima completa	55 %
Calidad y claridad del código	35 %
Reporte técnico	5 %
REPORTE_LM.md	5 %

Funcionalidad mínima completa (55 %)

Criterio	Puntos
Cumplimiento de los requisitos funcionales	4
Eficiencia y desempeño en general	1
Subtotal	5

Calidad y claridad del código (35 %)

Criterio	Puntos
Pertinencia del diseño	3
Pulcritud de la implementación	1
Pertinencia de la documentación del código	1
Subtotal	5